VITyarthi - Build Your Own Project

Number System Converter

A menu-driven, calculator-style tool for converting numbers between any two bases (2 to 36)

Student name	Krisha Chakraborty
Registration number	[fill in]
Programme	B.Tech Bioengineering, VIT
Course	CSE1021
Language / tools	Python 3, unittest, Git, GitHub
GitHub repository	[paste repository link]
Date of submission	[fill in]

Contents

1. Cover Page
2. Introduction
3. Problem Statement
4. Functional Requirements
5. Non-functional Requirements
6. System Architecture
7. Design Diagrams
8. Design Decisions and Rationale
9. Implementation Details
10. Screenshots / Results
11. Testing Approach
12. Challenges Faced
13. Learnings and Key Takeaways
14. Future Enhancements
15. References

2. Introduction

Number systems sit underneath everything in computing. Binary is how hardware stores data, hexadecimal is how programmers read memory and colours, and octal still appears in file permissions. Students constantly need to move between these representations, and doing it by hand is slow and error-prone.

This project is a Python command-line application that converts a number from any base between 2 and 36 to any other base in that range. It behaves like the mode-switching on a scientific calculator: the user converts a number, sees the result, and can immediately convert another one, review earlier conversions, or quit, all in the same session. Past conversions are also saved to a log file, so the history is still there the next time the program is run.

The code is split into three functional modules (input validation, the conversion engine, and history/logging) behind a small menu-driven interface, and is covered by 21 automated unit tests.

3. Problem Statement

Students and programmers often need to convert numbers from one base to another (binary, octal, decimal, hexadecimal and others). Many simple converters only support one fixed pair of bases, such as binary to decimal, and do not let the user carry on converting within the same session.

This project addresses that gap by providing a general converter that works between any two bases from 2 to 36, validates every input before converting it, remembers what has been converted, and lets the user repeat the process as often as they like.

Objectives

- Convert integers (positive and negative) between any two bases from 2 to 36.
- Reject invalid bases and invalid digits with a clear message instead of crashing or giving a wrong answer.
- Keep a history of conversions, viewable on demand and persisted between runs.
- Provide a simple, repeatable menu workflow.
- Build the solution as clean, separate modules with automated tests.

Scope

Command-line application; integer conversion only. Fractional numbers (for example 101.101 in binary) and a graphical interface are outside the current scope and are listed under future enhancements.

4. Functional Requirements

The project has three major functional modules, plus a menu interface that ties them together.

ID	Module	Requirement
FR1	Module 1: Validator	Check that the base is a whole number between 2 and 36.
FR2	Module 1: Validator	Check that every character of the number is a legal digit for its base (for example 9 is illegal in base 8); accept an optional leading minus sign; reject empty input.

ID	Module	Requirement
FR3	Module 2: Engine	Convert a number from any base to decimal.
FR4	Module 2: Engine	Convert a decimal integer to any base, including 0 and negative values.
FR5	Module 2: Engine	Combine FR3 and FR4 so a single call converts from any base to any base.
FR6	Module 3: History	Record each conversion with a timestamp, show the list on request, and clear it.
FR7	Module 3: History	Append each conversion to conversion_log.txt and reload it at start-up.
FR8	Interface	Menu loop: convert, view history, clear history, quit. Accept aliases such as binary, octal, decimal, hex.

Input / output structure

Input	Processing	Output
Menu choice (1 to 4)	Menu dispatch in main.py	The chosen action runs, then the menu is shown again
Number as text, source base, target base	Validate, convert to decimal, convert to target base	Converted number, plus the decimal value used along the way
Invalid base or digit	ValidationError raised and caught	Friendly error message; program continues

5. Non-functional Requirements

Requirement	How the project meets it
Usability	Menu-driven interface with clear prompts that list the accepted base names; results show the decimal equivalent so the user can follow the working.
Reliability	All user-facing work is wrapped in exception handling, so bad input or an unexpected fault returns the user to the menu instead of crashing. A failure to write the log file is reported as a warning and never stops the program.
Error handling strategy	A single custom exception, ValidationError, is raised for every expected problem and caught in one place with a friendly message. A general handler catches anything unexpected.
Maintainability	Three modules with one job each, docstrings and comments throughout, and marked extension points. The interface is separate from the conversion logic, so either can change without touching the other.
Scalability	The digit alphabet (0-9 then A-Z) is defined in one place; supporting more bases means changing that constant and the range limits.
Logging / monitoring	Every conversion is timestamped and appended to conversion_log.txt.

Requirement	How the project meets it
Performance and resource efficiency	Conversion takes a number of steps proportional to the number of digits. The log file is only touched if logging is switched on.
Security (input safety)	All input is validated before use, and nothing typed by the user is ever executed or evaluated.

6. System Architecture

The application has two layers. main.py is the interface layer: it prints the menu, reads input, and displays results. The converter package is the logic layer, split into three modules. The history module is the only part that touches the file system.

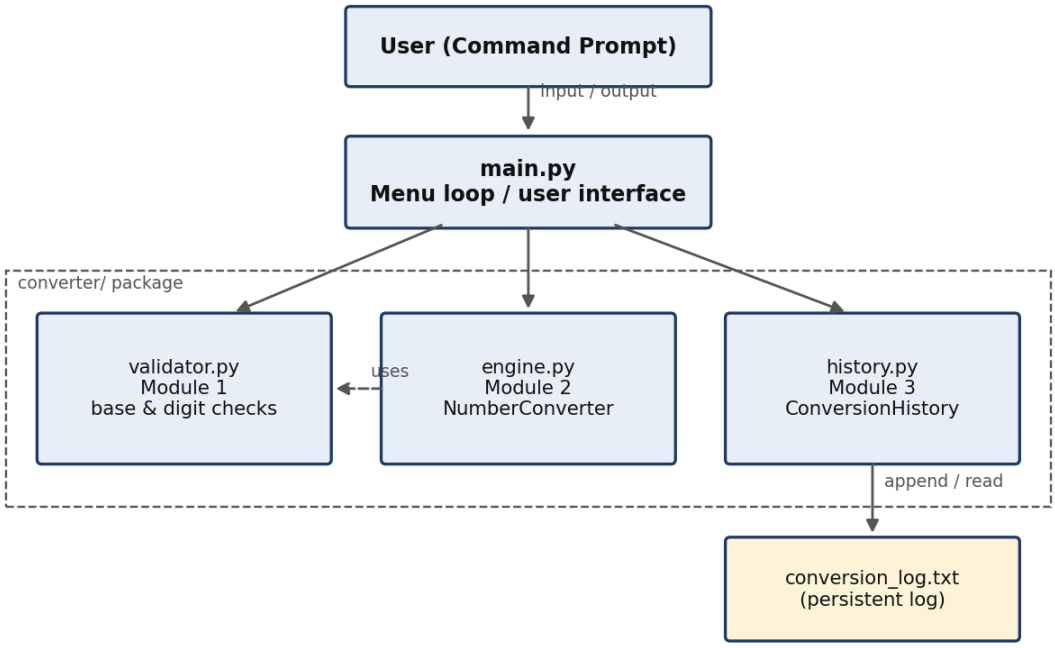


Figure 1: System architecture

7. Design Diagrams

7.1 Use case diagram

The only actor is the user. Every conversion includes an input-validation step.

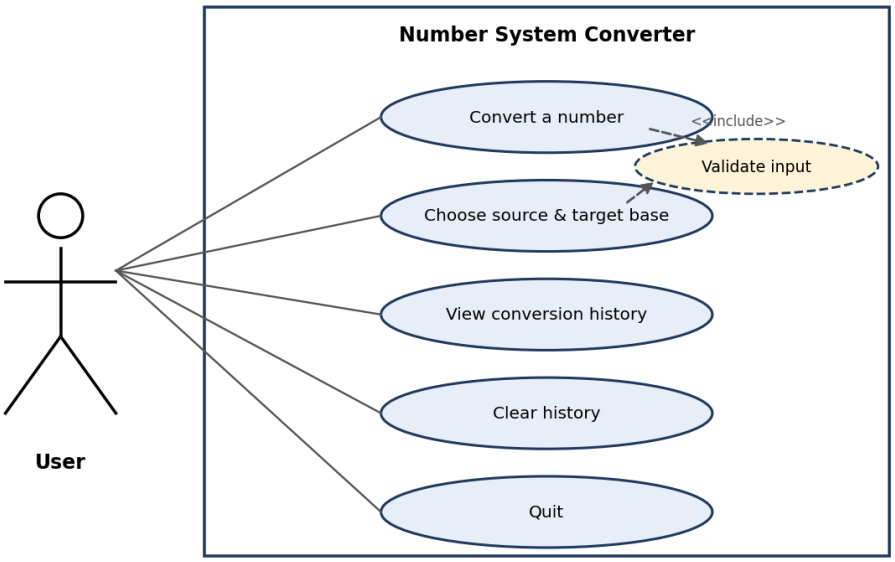


Figure 2: Use case diagram

7.2 Workflow diagram

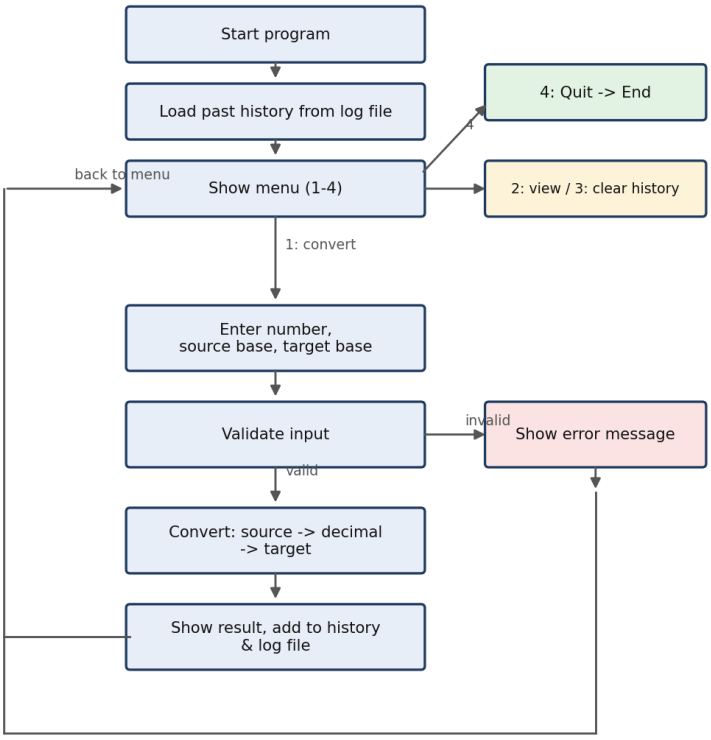


Figure 3: Workflow of a session

7.3 Sequence diagram

Sequence for a single successful conversion.

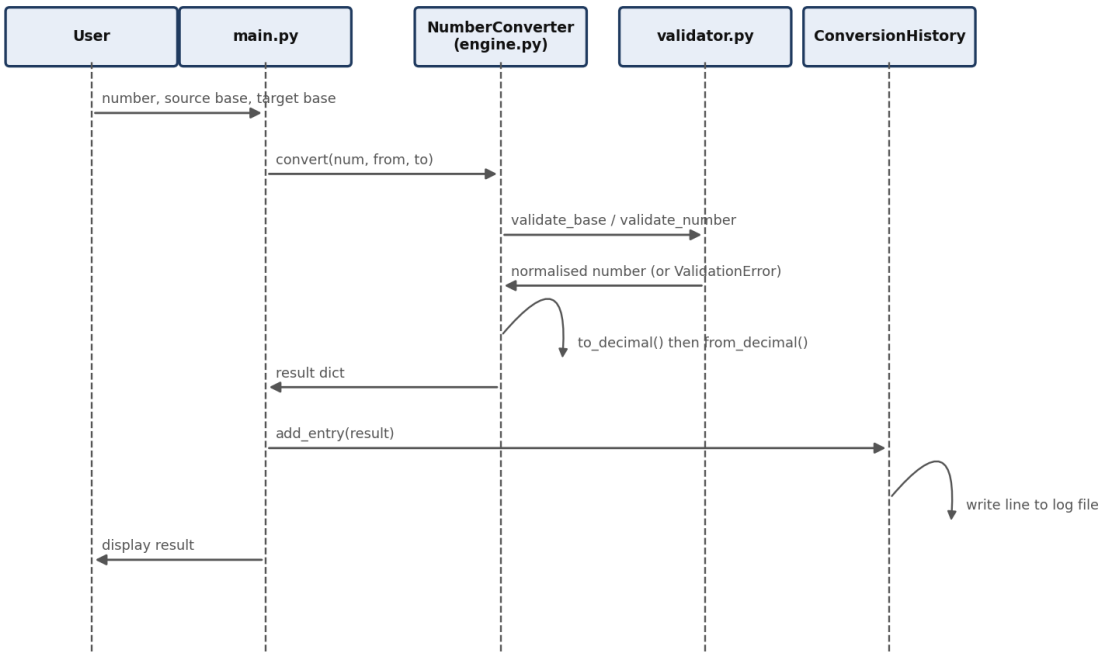


Figure 4: Sequence diagram for one conversion

7.4 Class / component diagram

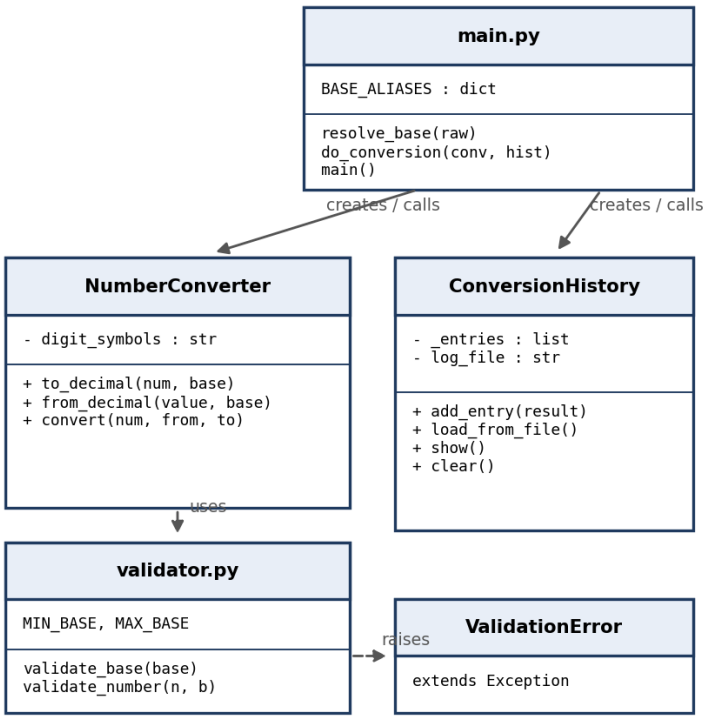


Figure 5: Class and component diagram

7.5 ER diagram

No database is used, so no ER diagram is needed. The only storage is a plain text log file, with one line per conversion in this format:

```
[2026-09-26 15:57:25] 1010 (base 2) -> A (base 16)
```

8. Design Decisions and Rationale

Decision	Rationale
Convert through decimal (source -> decimal -> target)	Only two algorithms are needed instead of one for every pair of bases. With 35 usable bases that would otherwise be more than a thousand pairs.
Separate validator, engine and history modules	Each file has one responsibility, which keeps the code readable, testable in isolation, and easy to extend.
Custom ValidationError	Expected problems are told apart from genuine bugs, and the interface can show a friendly message for one and a fallback message for the other.
Class-based NumberConverter and ConversionHistory	Groups related behaviour and data, and allows state such as caching to be added later without changing how main.py calls them.
Bases 2 to 36 with 0-9 then A-Z	This is the natural maximum, because 10 digits plus 26 letters give 36 symbols.
Plain text log file	Simple, human-readable, and needs no database for this scale.
Base aliases in main.py, not the engine	Names such as hex are a user-interface convenience, so the engine stays purely numeric.

9. Implementation Details

Project structure

```
number_converter/
|-- main.py                 # menu loop / user interface
|-- converter/
|   |-- __init__.py
|   |-- validator.py        # Module 1: input validation
|   |-- engine.py           # Module 2: base conversion
|   `-- history.py          # Module 3: history and logging
|-- tests/
|   |-- test_validator.py
|   `-- test_engine.py
|-- statement.md
`-- README.md
```

validator.py

Defines MIN_BASE = 2, MAX_BASE = 36 and the digit alphabet. validate_base() checks type and range. validate_number() upper-cases the input, strips whitespace, allows one leading minus sign, and checks every character against the first *base* symbols of the alphabet. All failures raise ValidationError.

engine.py

NumberConverter.to_decimal() walks the digits from left to right, updating result = result * base + digit. NumberConverter.from_decimal() repeatedly applies divmod(n, base), collects the remainders, and reverses them. Zero and negative values are handled explicitly. convert() chains the two and returns a dictionary holding the input, both bases, the decimal value and the output.

```
result = 0
for char in digits:
    value = self.digit_symbols.index(char)
```

```
            result = result * base + value          # e.g. "1010" in base 2 -> 10
```

history.py

ConversionHistory keeps a list of entries in memory. add_entry() timestamps a result and, if a log file was given, appends a line to it. load_from_file() reads those lines back with a regular expression when the program starts. show() and clear() serve the menu. File errors are caught so logging can never stop the program.

main.py

resolve_base() turns text such as hex or 16 into an integer base. do_conversion() asks for the number and both bases, calls the converter, prints the result and records it. main() creates the objects, loads any saved history, and runs the menu loop until the user chooses Quit.

10. Screenshots / Results

Replace each box below with a screenshot taken from Command Prompt while the program is running.

[Insert screenshot here: main menu and one conversion, e.g. 23 (decimal) to binary]

[Insert screenshot here: several conversions in one session, showing the menu looping back]

[Insert screenshot here: an invalid input, e.g. digit 9 in base 8, showing the friendly error message]

[Insert screenshot here: history view (option 2), and the 'Loaded ... conversion(s)' message on a second run]

[Insert screenshot here: unit tests: python -m unittest discover tests -v, ending with OK]

Sample results

Input	From base	To base	Output
1010	2	16	A
255	10	2	11111111
FF	16	8	377
-101	2	10	-5
23	10	2	10111

11. Testing Approach

Testing uses Python's built-in unittest module. There are 21 tests in two files, run with `python -m unittest discover tests -v`.

Test file / class	What it checks
test_validator: TestValidateBase (4)	Accepts bases 2, 16 and 36; rejects 1, 37 and non-integers.
test_validator: TestValidateNumber (6)	Accepts valid binary and hex (any letter case) and negatives; rejects illegal digits, empty input and a lone minus sign.
test_engine: TestToDecimal (4)	Binary, hex, octal and negative numbers convert to the right decimal.
test_engine: TestFromDecimal (4)	Decimal to binary and hex, zero in any base, and negatives.
test_engine: TestFullConvertRoundTrip (3)	Binary to hex in one call; base 7 to base 29 and back returns the original; invalid digits raise ValidationError.

The menu itself was also exercised by hand with valid input, invalid digits and invalid base names. [Add a line here about your own manual tests, for example repeated runs to check history

persistence.]

12. Challenges Faced

- **Indentation errors.** Mixing tabs and spaces in the same file produced repeated IndentationError messages that pointed at a line other than the real problem. The fix was to use spaces only, with four per level.
- **Changes not taking effect.** After editing, the terminal kept showing the old error, so the file being run was not the file being edited. Recreating the file inside the correct project folder and checking the folder path in the terminal against the editor's path resolved it.
- **Small typos causing NameError.** A misnamed parameter and a misspelled variable made names appear undefined. Reading the method signature line by line found them.
- **A safety net hiding the cause.** The general exception handler showed only a one-line message, which hid the traceback while debugging. Temporarily disabling it exposed the real error, and it was then restored for reliability.
- **Parsing the log file.** Reading history back required a regular expression matching the exact line format written by the logger.

13. Learnings and Key Takeaways

- Splitting a program into small modules with one job each makes it far easier to test and to debug.
- Validating input at the boundary, and raising one clear custom exception, keeps the rest of the code simple.
- Editor settings matter: consistent indentation and knowing exactly which file is being run save hours.
- Reading a traceback carefully, and knowing when to remove error-hiding code temporarily, is a core debugging skill.
- Writing tests early gives confidence that later changes, such as adding persistence, did not break existing behaviour.
- Using Git and a clear README from the start makes the project easy for someone else to run.

14. Future Enhancements

- Support fractional numbers (for example 101.101 in binary).
- Batch conversion of many numbers from a file.
- Extra aliases such as oct, and a help screen listing all supported bases.
- A graphical interface (Tkinter) or web front end reusing the same converter package.
- Show step-by-step working for each conversion, which would suit learners.
- Bitwise views such as two's complement for negative binary numbers.

15. References

- Python Software Foundation. *The Python Standard Library: unittest*. docs.python.org.
- Python Software Foundation. *The Python Tutorial*, sections on classes and errors and exceptions. docs.python.org.

- Course material for CSE1021, VIT.
- VITyarthi. *Build Your Own Project: General Project Instructions and Submission Guidelines*.
- [Add any other sources you actually used, such as a textbook or website on number-base conversion.]